

1. Introduction to Loops

In programming, loops are used to repeat a set of instructions multiple times.

Loops help:

- Reduce repetitive code
- Save time
- Improve efficiency
- Automate tasks

There are mainly two commonly used loops:

- for loop
 - while loop
-

2. What is a while Loop?

A **while loop** is used to repeatedly execute a block of code as long as a condition remains true.

The loop continues until the condition becomes false.

Syntax of while Loop

```
while condition:  
    statements
```

Example

```
i = 1  
  
while i <= 5:  
    print(i)  
    i += 1
```

Output

```
1  
2  
3  
4  
5
```

Explanation

- i starts from 1
 - Condition $i \leq 5$ is checked
 - If true, the loop executes
 - $i += 1$ increases value of i
 - Loop stops when i becomes 6
-

3. Working of while Loop

The while loop works in the following steps:

1. Initialize variable
 2. Check condition
 3. Execute statements
 4. Update variable
 5. Repeat until condition becomes false
-

Flow Diagram of while Loop

Start



Initialize Variable



Check Condition



Condition True → Execute Statements



Update Variable



Go Back to Condition



Condition False → Stop

4. Flow of Execution

Example:

```
count = 1
```

```
while count <= 3:  
    print("Hello")  
    count += 1
```

Execution Process

Step count Value Condition Output

1	1	True	Hello
2	2	True	Hello
3	3	True	Hello
4	4	False	Loop Stops

5. Difference Between for Loop and while Loop

for Loop

Used when repetitions are known

Works with sequences

Simpler syntax

Mostly used for fixed iterations

while Loop

Used when repetitions are unknown

Works with conditions

More flexible

Mostly used for condition-based repetition

6. Infinite Loops

An **infinite loop** occurs when the condition never becomes false.

Example

```
i = 1
```

```
while i <= 5:  
    print(i)
```

Problem

The value of i is never updated.

Condition always remains true.

Solution

`i += 1`

must be added inside the loop.

7. Loop Control Statements

Loop control statements help control loop execution.

7.1 break Statement

The break statement immediately stops the loop.

Example

`i = 1`

`while i <= 10:`

`if i == 5:`

`break`

`print(i)`

`i += 1`

Output

1

2

3

4

7.2 continue Statement

The continue statement skips the current iteration.

Example

```
i = 0

while i < 5:
    i += 1

    if i == 3:
        continue

    print(i)
```

Output

```
1
2
4
5
```

8. Nested while Loops

A loop inside another loop is called a nested loop.

Example

```
i = 1

while i <= 3:
    j = 1

    while j <= 2:
        print(i, j)
        j += 1

    i += 1
```

Output

```
1 1
1 2
2 1
2 2
3 1
3 2
```

9. Introduction to Logic Building

Logic building means developing the ability to think step-by-step to solve problems efficiently.

It is one of the most important skills in programming.

Why Logic Building is Important

Logic building helps programmers:

- Solve problems efficiently
 - Write optimized code
 - Improve debugging skills
 - Develop analytical thinking
 - Create real-world applications
-

10. Steps to Build Programming Logic

Step 1: Understand the Problem

Read the problem carefully.

Step 2: Identify Inputs and Outputs

Example:

- Input → Number
 - Output → Even or Odd
-

Step 3: Break Problem into Small Steps

Example:

1. Take number
 2. Divide by 2
 3. Check remainder
 4. Display result
-

Step 4: Write Algorithm

Create step-by-step solution.

Step 5: Convert into Program

Write actual code.

11. Dry Run Technique

A **dry run** means manually executing a program step-by-step to understand how it works.

Example

```
num = 1
```

```
while num <= 3:  
    print(num)  
    num += 1
```

Dry Run Table

Iteration	num	Value	Output
1	1	1	
2	2	2	
3	3	3	

12. Real-Life Examples of Logic Building

Logic building is used in:

- ATM systems
- Login systems
- AI systems
- Mobile apps
- Banking software
- Traffic signal systems

- E-commerce websites
-

13. Basic while Loop Programs

Program 1: Print Numbers from 1 to 10

```
i = 1

while i <= 10:
    print(i)
    i += 1
```

Program 2: Print Even Numbers

```
num = 2

while num <= 20:
    print(num)
    num += 2
```

Program 3: Sum of Numbers

```
i = 1
total = 0

while i <= 5:
    total += i
    i += 1

print("Sum =", total)
```

Output

```
Sum = 15
```

14. Number Logic Programs

Program 1: Reverse a Number

```
num = 1234
reverse = 0
```



```
while num > 0:
    digit = num % 10
    reverse = reverse * 10 + digit
    num = num // 10

print(reverse)
```

Output

4321

Program 2: Count Digits

```
num = 98765
count = 0

while num > 0:
    count += 1
    num = num // 10

print(count)
```

Output

5

15. Pattern Programs Using while Loop

Star Pattern

```
rows = 5
i = 1

while i <= rows:
    j = 1

    while j <= i:
        print("*", end=" ")
        j += 1

    print()
    i += 1
```

Output

```
*  
* *  
* * *  
* * * *  
* * * * *
```

16. Common Errors in while Loops

1. Forgetting Variable Update

This causes infinite loops.

2. Wrong Condition

while i > 5:

may never execute if i starts below 5.

3. Indentation Errors

Wrong indentation causes syntax problems.

17. Advantages of while Loops

- Flexible looping
- Useful when iterations are unknown
- Easy condition-based repetition
- Suitable for real-world systems
- Helpful in menu-driven programs

18. Real-Life Applications

while loops are used in:

- Password validation systems
- ATM machines
- Game loops

- Chat applications
- AI systems
- Login systems
- Data processing software